

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Multidisciplinary Project (CO3107)

Project Report - Semester 252

“RubyAlert: An Intelligent AIoT-Driven Laboratory Monitoring and Early Warning System”

Instructor(s): Dr. Vo Tuan Binh

Students: Le Hoang Chi Vi – 2353336 (*Group CC01, Team 29*)

HO CHI MINH CITY, MAY 2026

List of Figures

3.1	Vector Diagram of the RubyAlert Multi-tier AIoT Architecture	10
3.2	Central Local Core Edge Algorithm Execution Flowchart	12
5.1	Web Dashboard State: MQTT Connection Initialized	15
5.2	Web Dashboard State: Normal Safe Telemetry Stream	16
5.3	Web Dashboard State: Critical Threshold Breach Alert for Gas Leakage .	16
5.4	Telegram Chatbot Workflow: First Telemetry Stream	17
5.5	Telegram Chatbot Workflow: AI Function Calling for Device Control . .	17
5.6	Telegram Chatbot Workflow: Proactive Warning Alerts	18

List of Tables

3.1	ESP32 Hardware Pin Connections	9
5.1	RubyAlert System Verification Matrix	18

Contents

List of Figures	1
List of Tables	2
1 Introduction	5
1.1 Background of the Study:	5
1.2 Research Objectives:	5
1.3 Scope and Limitations	6
2 Foundational Knowledge	7
2.1 Internet of Things (IoT) Architecture	7
2.2 Communication Protocol Selection	7
2.3 Core Hardware Modules	7
2.4 Artificial Intelligence in IoT (AIoT)	8
3 System Design	9
3.1 Hardware Schematic & Connection Specifications	9
3.2 System Architecture Vector Drawing	9
3.3 Centralized Edge Automation Algorithms	10
3.3.1 Hysteresis Automation	10
3.3.2 Rate of Change (RoC) Detection	10
3.3.3 Sensor Fault-Tolerance & Hardware-Safe Fallbacks	11
3.3.4 Manual Override Priority	11



3.3.5	Algorithmic Flowchart	11
4	Methodology	13
4.1	Firmware and Server Development	13
5	Experimental Results	15
5.1	Visual Verification & Dashboard Evidence	15
5.2	Telegram Conversational Log Evidence	16
5.3	System Verification and Test Cases	18
6	Conclusion	19
6.1	Key Contributions	19
6.2	Future Roadmap	19

Chapter 1

Introduction

Open-Source Project Repository: The entire codebase, including the simulated multi-model edge server, firmware scripts, and responsive dashboard files, is hosted publicly to support academic reproducibility and collaboration: <https://github.com/ukulevi/rubyalert-aiot-lab>

1.1 Background of the Study:

Laboratory safety represents one of the most critical aspects of academic and industrial research environments. High stress operations involving volatile solvents, pressurized gas channels, active electrical grids, and automated heaters are perpetually performed. In Vietnam, past fire anomalies and harmful chemical inhalations in research laboratories highlight the severe deficiency of proactive, autonomous safety architectures. Conventional IoT implementations are largely static, displaying telemetry on isolated physical screens or simple web views without remote control loops or natural-language interaction capacities.

The rapid rise of the *Artificial Intelligence of Things (AIoT)* allows engineers to merge local microcontrollers with state-of-the-art Large Language Models (LLMs) like Google Gemini. This paradigm shifts safety monitoring from a classic passive/reactive rule-check model to a cognitive, self-governing ecosystem.

1.2 Research Objectives:

The primary objectives of this study are defined as:

1. **Real-time Telemetry Collection:** Design a physical sensory node using the ESP32 to track Temperature, Humidity, and Gas/Smoke parameters via DHT22 and MQ-2 inputs.

2. **High-Availability Dashboard:** Build a sleek Web Dashboard based on modern design systems displaying low-latency telemetry streams via WebSockets.
3. **Bi-directional Alerting Infrastructure:** Configure a Telegram Chatbot delivering unified notifications to operators while accommodating remote control parameters.
4. **Cognitive AI Orchestration:** Program Google Gemini AI to act as a reactive lab assistant, execute hardware configurations via Function Calling, and autonomously initiate emergency controls (e.g., fan actuation) when relative risks emerge.

1.3 Scope and Limitations

The research parameters are structured as:

- **Scope:** Deployment is confined to a singular, standard university chemistry/IoT laboratory. Actuation is simulated through a 5V relay driving a DC ventilation fan, supplemented with an active warning buzzer.
- **Limitations:** Messages are transmitted over the public `broker.emqx.io` MQTT Broker. Telemetry parameters are focused on relative hazard thresholds rather than absolute molecular spectrometry measurements.

Chapter 2

Foundational Knowledge

2.1 Internet of Things (IoT) Architecture

The system layout conforms to the standard three-tier IoT architectural standard:

1. **Perception Layer:** Comprises the ESP32 microcontroller interfaced with the DHT22 and MQ-2 physical sensors, a 5V relay, and a piezoelectric buzzer.
2. **Network Layer:** Uses MQTT over Wi-Fi for telemetry delivery, plus HTTPS and WSS channels for AI cloud calls and web synchronization.
3. **Application Layer:** Entails the central Python Edge server, the Telegram Chatbot daemon, the HTML5 Web Dashboard, and the Google Gemini NLU API.

2.2 Communication Protocol Selection

For physical-node to edge-server exchange, **MQTT (Message Queuing Telemetry Transport)** was chosen over standard HTTP due to its low header footprint (2 bytes minimum vs. kilobytes in HTTP headers), minimal network bandwidth, and native publish/subscribe messaging, which is ideal for real-time telemetry systems.

2.3 Core Hardware Modules

- **ESP32 MCU:** A dual-core 32-bit CPU running at 240MHz, offering integrated Wi-Fi/Bluetooth, extensive hardware GPIO pins, and an internal 12-bit ADC essential for reading the analog voltage of the MQ-2 sensor.
- **DHT22 Digital Sensor:** Utilizes a capacitive humidity sensor and a thermistor to measure surrounding air, outputting a calibrated digital signal on a single-wire bus (Temperature: -40 to $80^{\circ}\text{C} \pm 0.5^{\circ}\text{C}$; Humidity: 0 - 100% RH $\pm 2\%$).

- **MQ-2 Gas Sensor:** Features a tin dioxide (SnO_2) semiconductor layer. When exposed to combustible gases (LPG, Propane, CO, Methane, Smoke), the surface conductivity increases, yielding an analog output voltage directly proportional to gas density.

2.4 Artificial Intelligence in IoT (AIoT)

Rather than executing static scripts, the RubyAlert system embeds **Google Gemini AI** to introduce cognitive reasoning at the edge. By utilizing **Function Calling**, the LLM maps natural-language instructions to automated backend commands. The **3-Tier AI System Model** implemented in this work is structured as follows:

- **Tier 1: Reactive Assistant:** Generates environmental audit reports and answers descriptive lab queries.
- **Tier 2: Agentic Actor:** Parses conversational intent and actuates relays (e.g., turning on the fan when the user states “it feels hot in here”).
- **Tier 3: Autonomous Supervisor:** Evaluates early hazard escalation and autonomously acts before standard local alarms trigger.

Chapter 3

System Design

3.1 Hardware Schematic & Connection Specifications

The ESP32 pin configuration is systematically mapped in Table 3.1.

Component	Sensor Pin	ESP32 GPIO	Signal Type
DHT22	OUT	GPIO 4	Digital I/O
MQ-2	AO (Analog Out)	GPIO 34	Analog Input (ADC1)
Relay Module	IN	GPIO 27	Digital Output
Active Buzzer	Positive (+)	GPIO 26	Digital Output

Table 3.1: ESP32 Hardware Pin Connections

3.2 System Architecture Vector Drawing

To represent the data pathways and physical hardware nodes within RubyAlert, Figure 3.1 details the complete AIoT telemetry network.

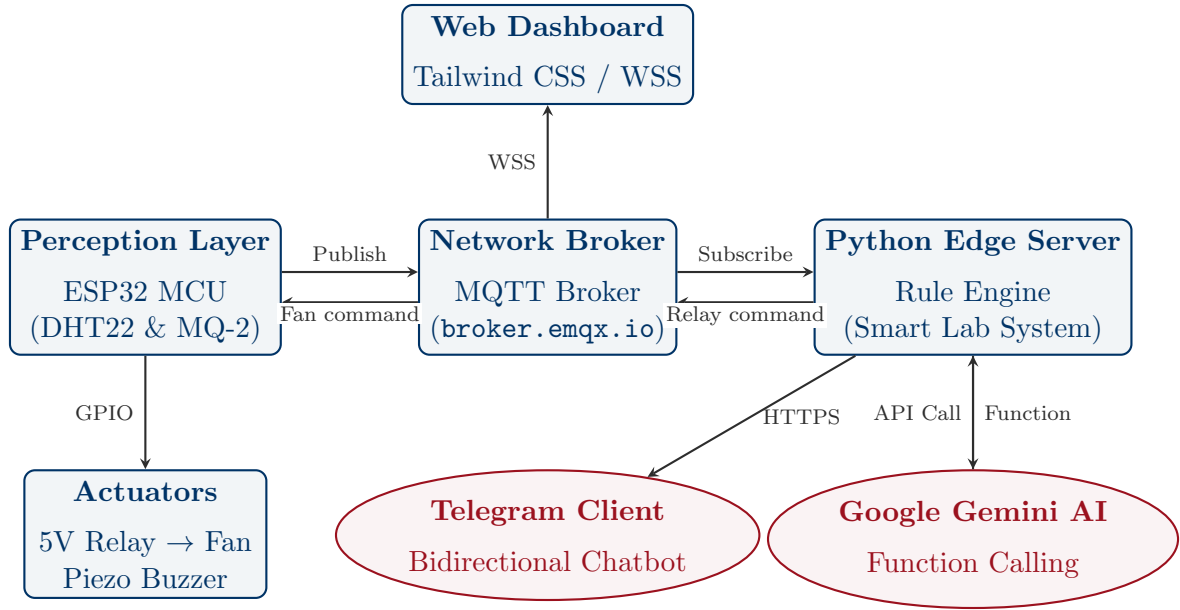


Figure 3.1: Vector Diagram of the RubyAlert Multi-tier AIoT Architecture

3.3 Centralized Edge Automation Algorithms

To safeguard hardware integrity and ensure rapid reaction times, localized edge logic overrides cloud loops:

3.3.1 Hysteresis Automation

To relieve stress on the physical relay components caused by high-frequency switching near critical thresholds, a hysteresis band is enforced:

$$T_{\text{activation}} = 35^{\circ}\text{C} \quad (3.1)$$

$$T_{\text{deactivation}} = 32^{\circ}\text{C} \quad (3.2)$$

The actuator stays active until telemetry falls below the safe deactivation limits.

3.3.2 Rate of Change (RoC) Detection

Sudden thermal spikes indicative of chemical fires are computed over localized histories:

$$\text{RoC} = T_{\text{current}} - T_{t-10s} \quad (3.3)$$

If $\text{RoC} \geq 3^{\circ}\text{C}$ in under 10 seconds, the active buzzer and high-priority SMS notifications are immediately fired.

3.3.3 Sensor Fault-Tolerance & Hardware-Safe Fallbacks

If a sensor is physically severed, the ESP32 intercepts empty (NaN) data. The edge server identifies this, immediately suspending automated script blocks to prevent false actuation, and activates a safe default state.

3.3.4 Manual Override Priority

To respect human authority, manual toggle commands issued via the Web Dashboard or Telegram raise a `manual_override` flag, temporarily pausing the automatic rule engine.

3.3.5 Algorithmic Flowchart

The logic execution path of the localized edge computing platform is mapped in Figure 3.2.

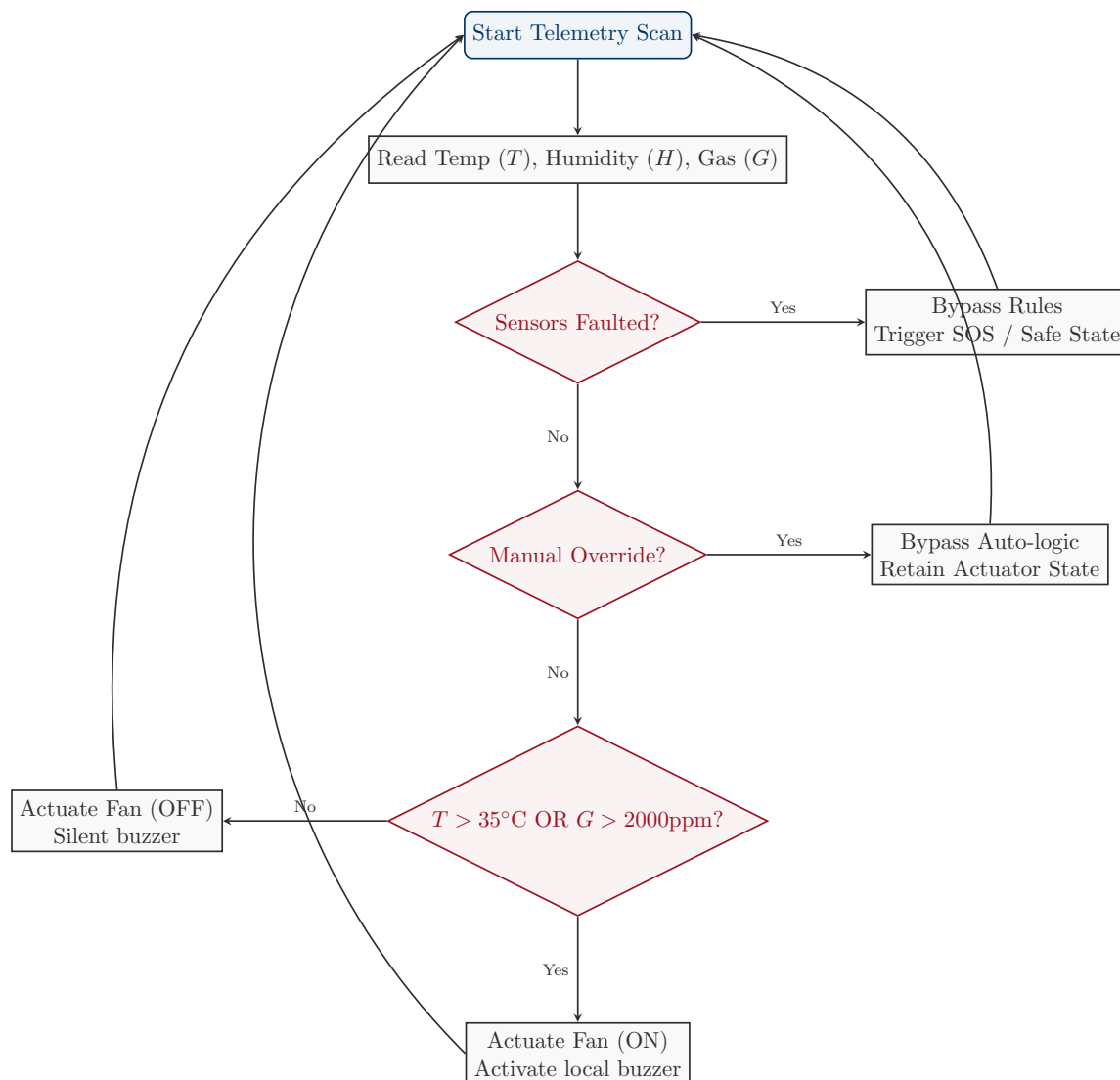


Figure 3.2: Central Local Core Edge Algorithm Execution Flowchart

Chapter 4

Methodology

4.1 Firmware and Server Development

The ESP32 node is programmed using **MicroPython** for modular, rapid scripting. The central backend server is implemented in **Python 3** utilizing the Google GenAI SDK and Paho MQTT library.

An illustrative example of the centralized server-side telemetry ingest and rule engine is shown below:

Code:

```
# Local Edge Rule Engine & Hysteresis Implementation
def evaluate_telemetry_rules(temp, humidity, gas):
    global fan_state, manual_override

    if manual_override:
        print("Manual Override active. Bypassing automation
              rules.")
        return

    # Standard thresholds
    TEMP_TRIGGER = 35.0
    TEMP_SAFE = 32.0
    GAS_TRIGGER = 2000.0
    GAS_SAFE = 1200.0

    # Hysteresis Automation
    if temp > TEMP_TRIGGER or gas > GAS_TRIGGER:
        if fan_state == "OFF":
            set_fan_state("ON")
            send_telegram_sos("CRITICAL: Threshold exceeded.
                              Activating fan.")
    elif temp < TEMP_SAFE and gas < GAS_SAFE:
        if fan_state == "ON":
            set_fan_state("OFF")
            send_telegram_sos("INFO: Environment safe.
                              Deactivating fan.")
```

The corresponding MQTT JSON format published by the ESP32 node is parsed as:

Output:

```
Received telemetry on topic:  rubyalert/lab_1/telemetry
Payload:
{
  "temperature":  28.5,
  "humidity":    65.2,
  "gas":         850.0
}
Status:  Environment safe.  Fan state:  OFF.
```

Chapter 5

Experimental Results

5.1 Visual Verification & Dashboard Evidence

To provide empirical evidence of the system in active runtime, the physical deployment and rendering of the Web Dashboard are demonstrated across the states shown in Figures 5.1–5.3.

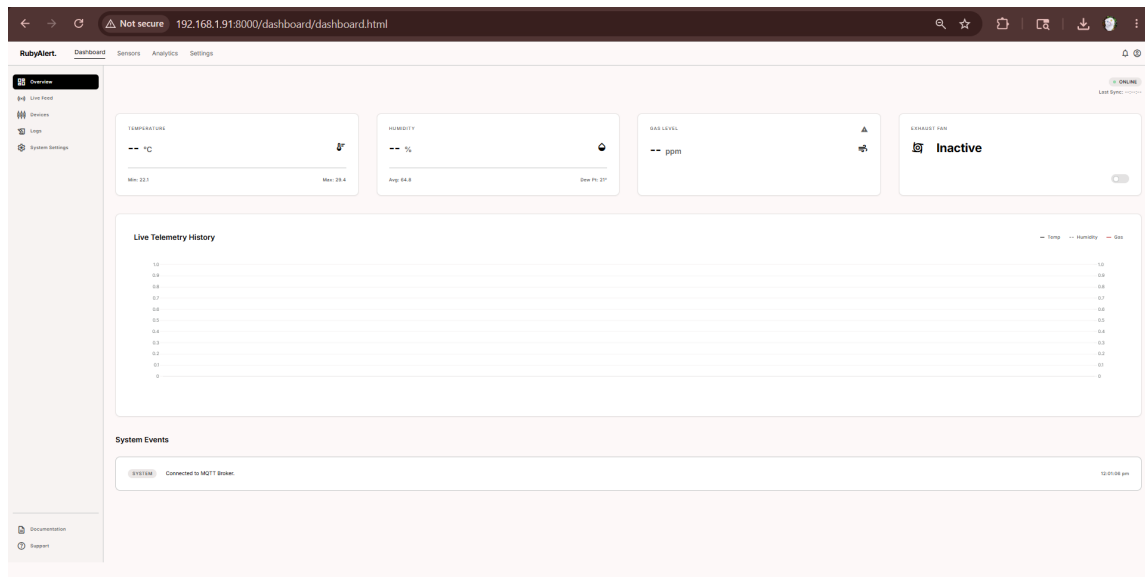


Figure 5.1: Web Dashboard State: MQTT Connection Initialized

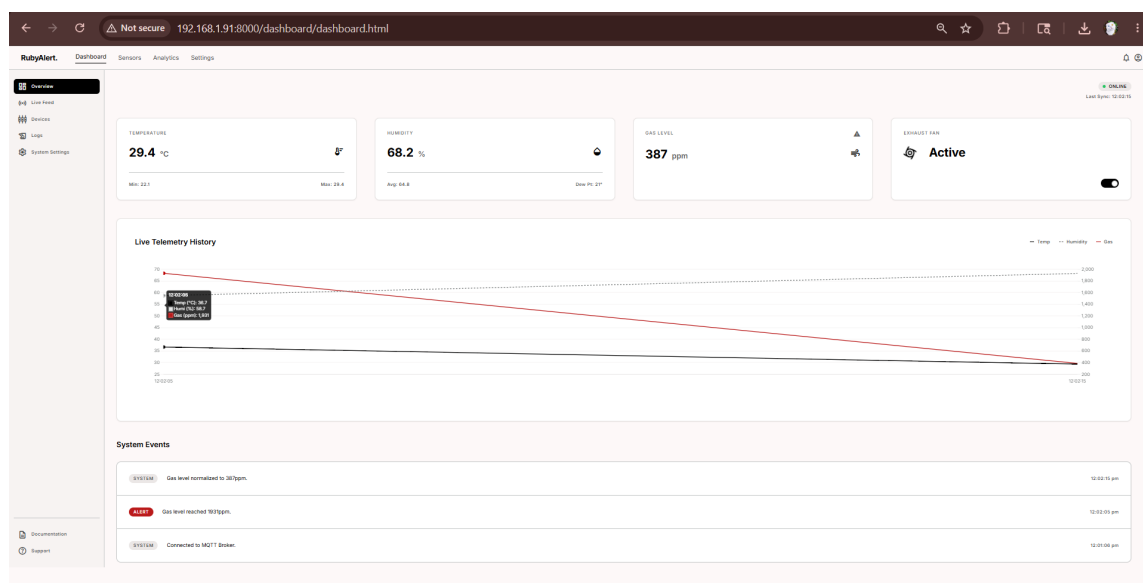


Figure 5.2: Web Dashboard State: Normal Safe Telemetry Stream

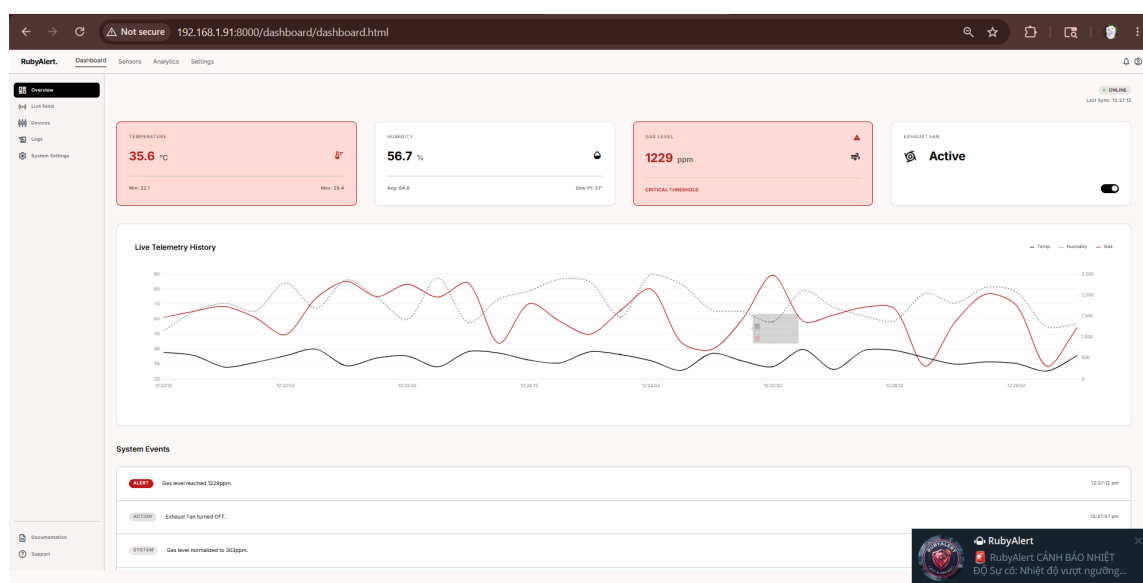


Figure 5.3: Web Dashboard State: Critical Threshold Breach Alert for Gas Leakage

5.2 Telegram Conversational Log Evidence

The actual conversation logs captured on a mobile device illustrating bidirectional control, telemetry requests, and proactive early warning notifications are documented in Figures 5.4–5.6.

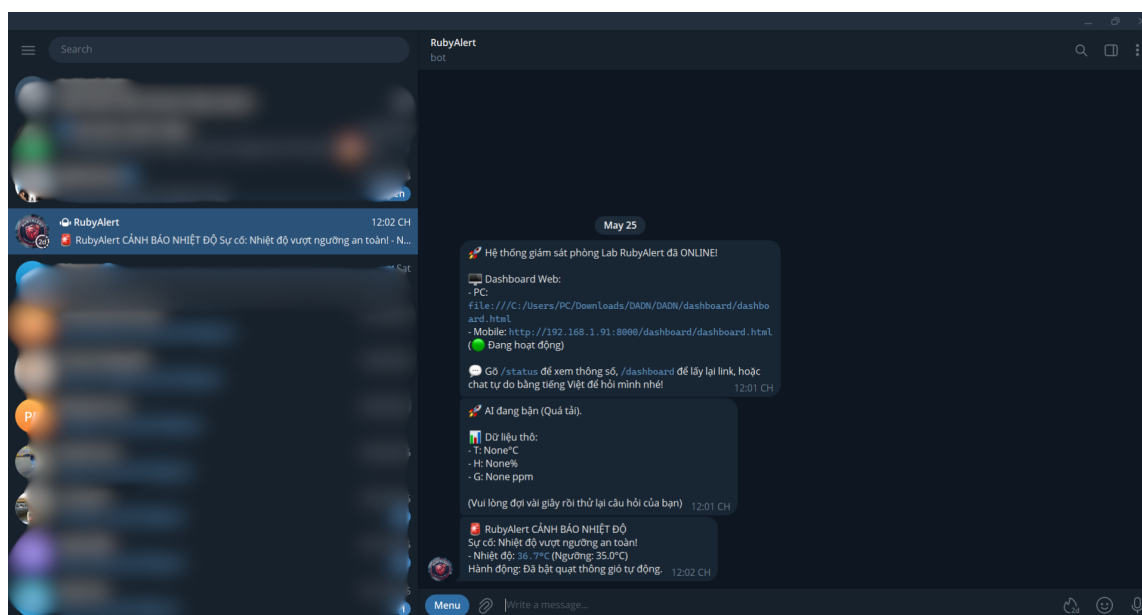


Figure 5.4: Telegram Chatbot Workflow: First Telemetry Stream

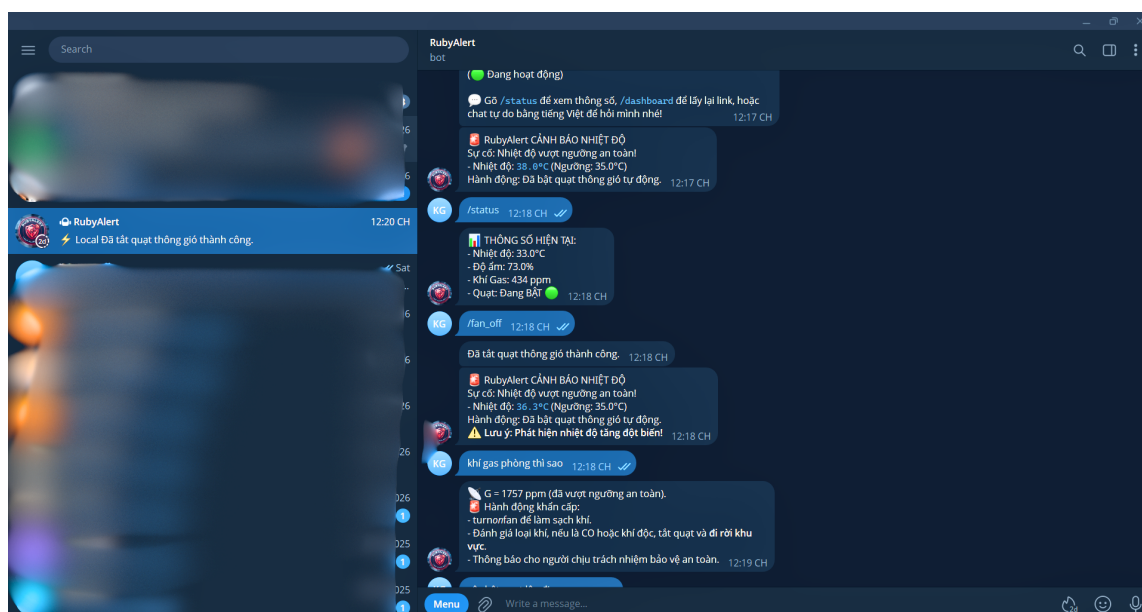


Figure 5.5: Telegram Chatbot Workflow: AI Function Calling for Device Control

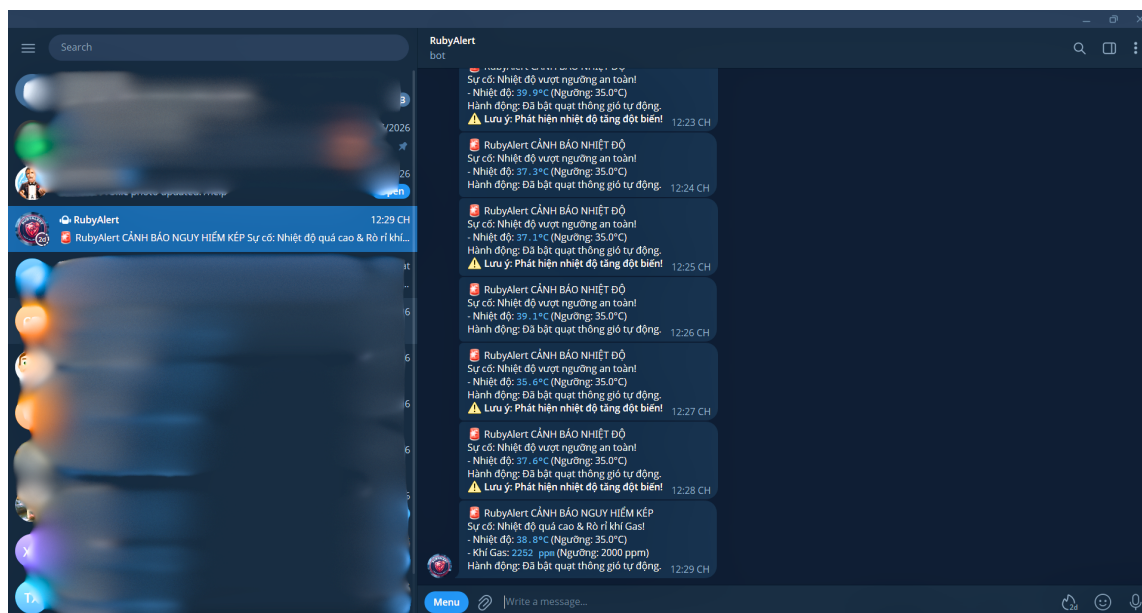


Figure 5.6: Telegram Chatbot Workflow: Proactive Warning Alerts

5.3 System Verification and Test Cases

The integrated hardware and edge server were subjected to 12 rigorous evaluation scenarios. The testing results are presented in Table 5.1.

ID	Scenario	Action	Expected Response	Actual Response	Status
TC01	Boot	Start server	Establish MQTT broker link	Connected to port 1884	Pass
TC02	Overheat	Telemetry $T = 38^{\circ}\text{C}$	Fan turns ON; Telegram SOS alert	Actuator active; alert sent	Pass
TC03	Hysteresis	Telemetry $T = 33^{\circ}\text{C}$	Fan remains ON	Fan active (No chattering)	Pass
TC04	Cooling	Telemetry $T = 30^{\circ}\text{C}$	Fan turns OFF	Fan deactivated at 32C	Pass
TC05	Gas Leak	Telemetry $G = 2200\text{ppm}$	Buzzer active; Fan turns ON	Actuators fired; SOS sent	Pass
TC06	Early Leak	Telemetry $G = 1300\text{ppm}$	AI Tier 3 turns on fan	AI resolved state; Fan ON	Pass
TC07	Command	Send <code>/status</code>	Send telemetry message	Real-time stats returned	Pass
TC08	NLP command	Send "turn on fan"	Gemini function call <code>set_fan</code>	Tool executed successfully	Pass
TC09	Filter	Send off-topic query	AI rejects prompt	Refusal prompt triggered	Pass
TC10	Report	Send <code>/report</code>	AI generates safety PDF report	Multi-paragraph PDF report	Pass
TC11	Override	Web Toggle pressed	Halts auto-rules	Manual control verified	Pass
TC12	Rate Limit	Bombard API with requests	Graceful fallback	Fallback text return	Pass

Table 5.1: RubyAlert System Verification Matrix

Chapter 6

Conclusion

6.1 Key Contributions

The **RubyAlert** project successfully validated that low-cost microcontrollers can be elevated into intelligent, cognitive safety platforms. The major contributions of this work are:

1. **Intelligent Edge Fallbacks:** Integration of local rule-based safety controls (Hysteresis, Rate of Change) overriding cloud loops during outages.
2. **Cognitive AIoT Architecture:** Development of a 3-tier AI system leveraging Google Gemini for reactive assistance, conversational actions, and autonomous early hazard prevention.
3. **High-Availability Design:** Implementations of hybrid keyword/AI processing, token limits, and AI cooldowns to operate 24/7 at zero operational cost.

6.2 Future Roadmap

Future extensions of the RubyAlert platform will focus on:

- **Private Broker Security:** Transitioning to a local Mosquitto MQTT Broker equipped with TLS/SSL encryption and password security.
- **Persistent Logging:** Integrating an InfluxDB database to store telemetry histories for long-term regression modeling.
- **On-board Diagnostics:** Wiring a physical SSD1306 OLED screen to the ESP32 to display sensor stats during network disconnections.

Bibliography

- [1] Google Gemini API documentation, Available online: <https://ai.google.dev/docs>.
- [2] OASIS Standard, MQTT Version 5.0, Available online: <https://mqtt.org/>.
- [3] Espressif Systems, ESP32 Technical Reference Manual, Available online: <https://espressif.com/>.
- [4] Le Hoang Chi Vi, RubyAlert AIoT Laboratory Monitoring and Warning System Source Code, GitHub Repository, 2026. Available online: <https://github.com/ukulevi/rubyalert-aiot-lab>.